WEEK-4

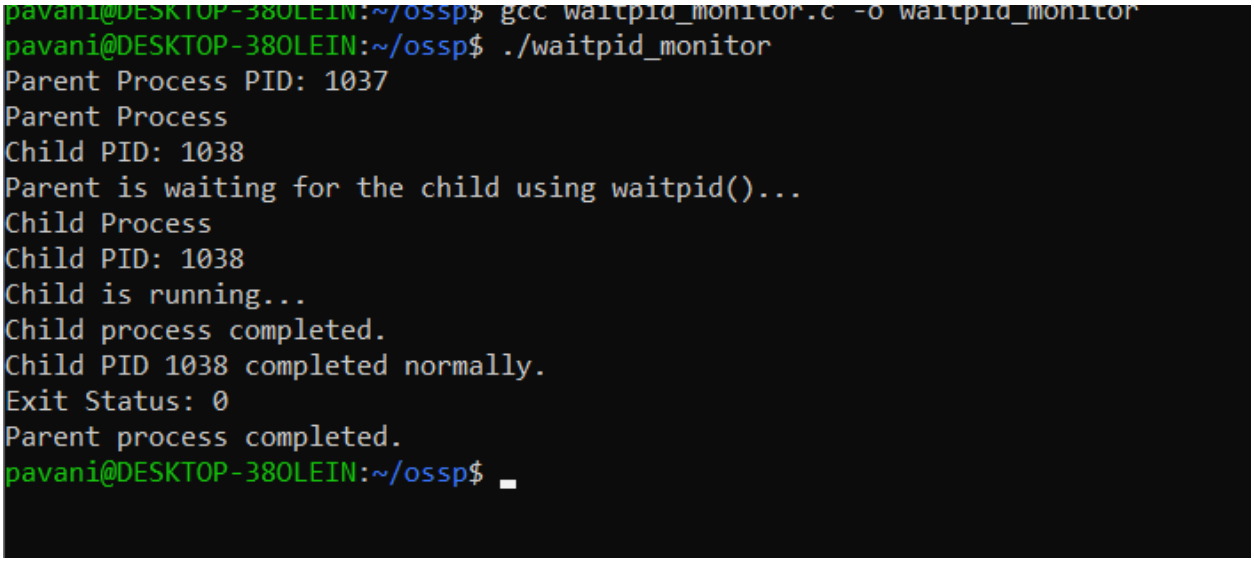
TASK-1 : Process Synchronization and Child Process Monitoring Using waitpid()

pavani@DESKTOP-38OLEIN: ~/ossp

GNU nano 8.7.1 waitpid_monitor.c *

```c
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("Parent Process PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Child is running...\n");

        sleep(3);

        printf("Child process completed.\n");
        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("Child PID: %d\n", pid);

        printf("Parent is waiting for the child using waitpid()...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status))
        {
            printf("Child PID %d completed normally.\n", pid);
            printf("Exit Status: %d\n", WEXITSTATUS(status));
        }

        printf("Parent process completed.\n");
    }

    return 0;
}
```

OUTPUT:



REPORT:

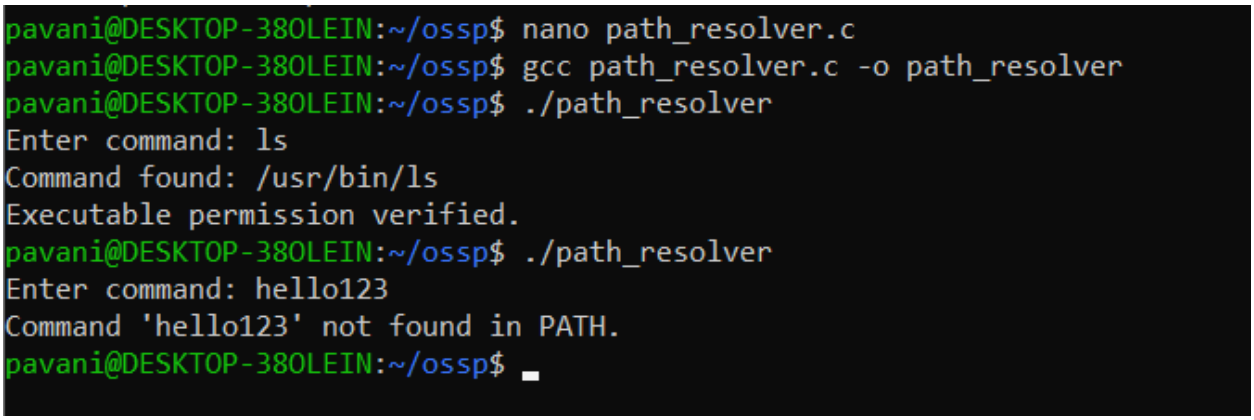
I created a parent and child process using fork() and used waitpid() to synchronize the child process with the parent. From the output, I observed that the parent waited until the child process completed and then checked its exit status. I understood how waitpid() helps the parent monitor and synchronize with a particular child process.

Task-2:

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

```
 GNU nano 8.7.1                                  path_resolver.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

#define MAX_PATH 4096

int main()
{
    char *path;
    char *path_copy;
    char *directory;
    char command[100];
    char full_path[MAX_PATH];

    printf("Enter command: ");
    scanf("%99s", command);

    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable not found.\n");
        return 1;
    }

    path_copy = strdup(path);

    if (path_copy == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    directory = strtok(path_copy, ":");

    while (directory != NULL)
    {
        snprintf(full_path, sizeof(full_path), "%s/%s", directory, command);

        if (access(full_path, X_OK) == 0)
        {
            printf("Command found: %s\n", full_path);
            printf("Executable permission verified.\n");

            free(path_copy);
```

OUTPUT:



Report:

I implemented a program to search for commands using the PATH environment variable. The program searches the directories in PATH and checks whether the command exists and has executable permission. I tested the program using the ls command and it successfully located the executable. I also tested a missing command to check the error handling. I understood how the shell searches for executable commands using the PATH variable.

Task-3:

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

pavani@DESKTOP-38OLEIN: ~/ossp

GNU nano 8.7.1 variable_expansion.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_OUTPUT 500

void expand_variables(char *input, char *output)
{
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_OUTPUT - 1)
    {
        if (input[i] == '$')
        {
            i++;

            if (input[i] == '{')
            {
                i++;
                char variable[100];
                int k = 0;

                while (input[i] != '\0' && input[i] != '}' && k < 99)
                {
                    variable[k++] = input[i++];
                }

                variable[k] = '\0';

                if (input[i] == '}')
                    i++;

                char *value = getenv(variable);

                if (value != NULL)
                {
                    int m = 0;

                    while (value[m] != '\0' && j < MAX_OUTPUT - 1)
                    {
                        output[j++] = value[m++];
                    }
                }
                else
                {
```

OUTPUT:

```
pavani@DESKTOP-38OLEIN:~/ossp$ gcc variable_expansion.c -o variable_expansion
pavani@DESKTOP-38OLEIN:~/ossp$ ./variable_expansion
Enter a string: Hello $USER
Expanded string: Hello pavani
pavani@DESKTOP-38OLEIN:~/ossp$ ./variable_expansion
Enter a string: $HOME
Expanded string: /home/pavani
pavani@DESKTOP-38OLEIN:~/ossp$
```

Report:

I implemented variable expansion in the shell by detecting variable references using $ and retrieving their values from the environment. I tested defined and undefined variables and observed that defined variables are replaced with their actual values, while undefined variables are reported. I understood how environment variables can be expanded and updated in the input string.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logical

Task-4:

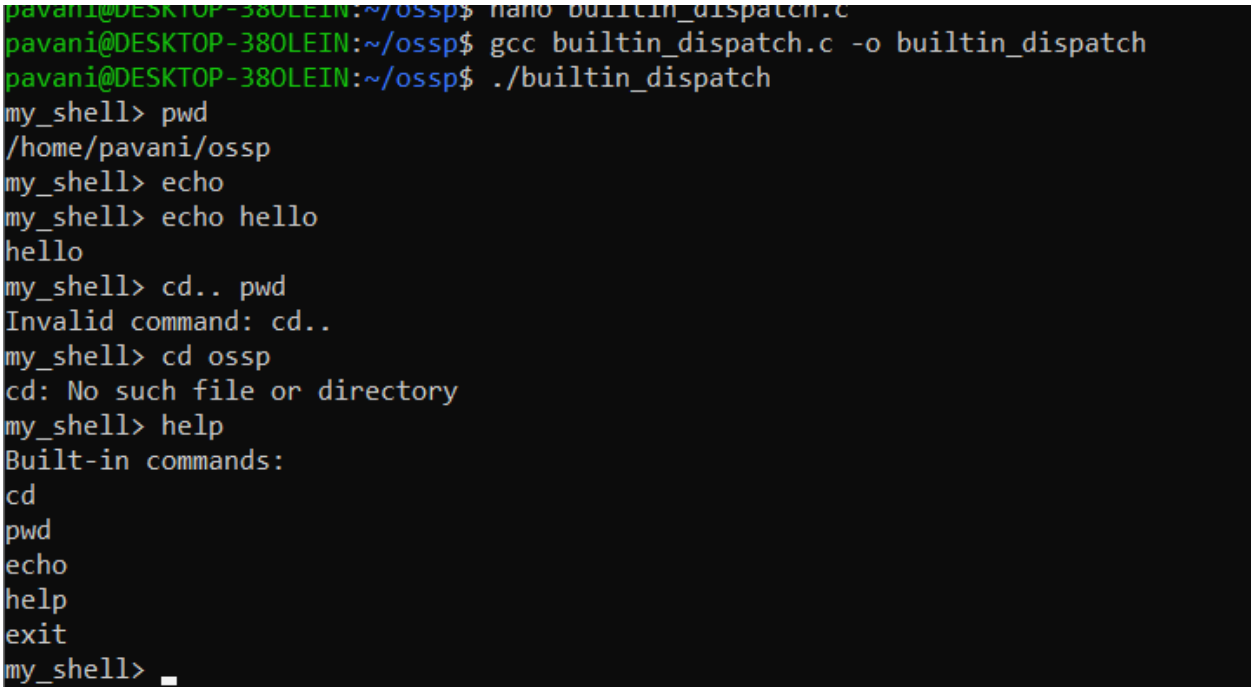
To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

pavani@DESKTOP-38OLEIN: ~/ossp

GNU nano 8.7.1 builtin_dispatch.c *

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100

void builtin_cd(char *args)
{
    if (args == NULL)
    {
        printf("cd: missing argument\n");
        return;
    }
    if (chdir(args) != 0)
    {
        perror("cd");
    }
}

void builtin_pwd()
{
    char path[1024];

    if (getcwd(path, sizeof(path)) != NULL)
    {
        printf("%s\n", path);
    }
}

void builtin_echo(char *args)
{
    if (args != NULL)
    {
        printf("%s\n", args);
    }
}

void builtin_help()
{
    printf("Built-in commands:\n");
    printf("cd\n");
    printf("pwd\n");
    printf("echo\n");
    printf("help\n");
    printf("exit\n");
}
```

OUTPUT:



Report:

I implemented a dispatch system for built-in shell commands such as cd, pwd, echo, help, and exit. I tested the commands and also checked an invalid command to verify error handling. I understood that built-in commands are executed within the shell process and that commands like cd can maintain the shell's current state.